Predicting flow with Back Propagation Neural Networks

F328799

Contents

Chapter 0

Outline

The aim of this project is to predict the flow-rate of the River Ouse at Skelton in advance to predict the flood potential. We will use a Back Propagation Neural Network to predict the flow-rate of the river, using data from the surrounding catchment area. This data includes river flow-rate from Skip Bridge, Crakehill and Westwick, as well as rainfall data from East Cowton, Arkengarthdale, Snaizeholme and Malham Tarn. The data was collected between 01/01/1993 to 31/12/1996.

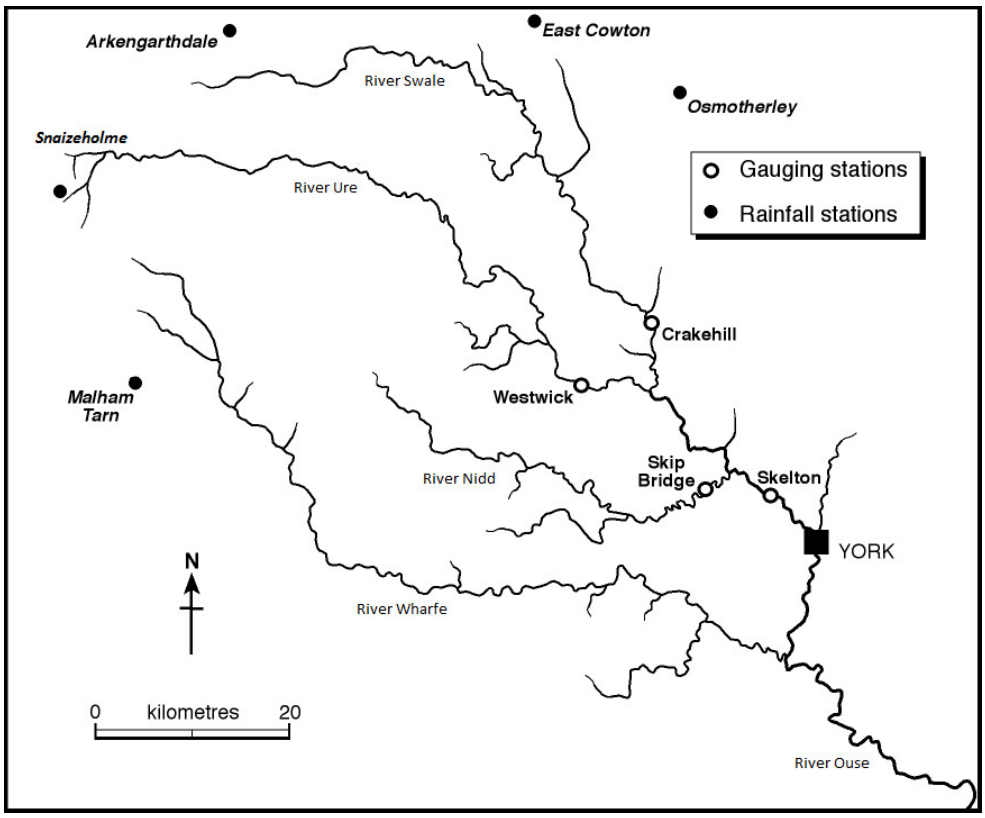


Figure 0.0.1: River Ouse Catchment

The sections of this report will include:

1. Data Pre-processing
2. Implementing the Neural Network
3. Model Training
4. Evaluation

0.1 System Environments

The ANN will be programmed using python 3.13.2 with a conda virtual environment. The libraries used will include:

- numpy
- pandas, sqlalchemy

- openpyxl
- sqlite3
- matplotlib

The data will be stored in an SQLite database, this is for ease of access and to keep the data in a single location, with concise SQL commands to access the data.
The data will be split into training and testing data, with 60% of the data used for training, 20% used for testing and 20% for evaluation.

Chapter 1

Data Pre-processing

1.1 Importing data

The first step is to import the data from the give xlsx file into the database. This will be done using the pandas library to read the data from the xlsx file and then using the sqlalchemy library to write the data to the database. For simplicity, I created a second xlsx file without the first row of headers, this is so that the data can be read into the database without any issues.

This code shows how I handled the non-numerical data in the xlsx file. It is then imported into the database using the sqlalchemy library. Line 16 is used to remove non-numerical data when importing into the database. This is to avoid any type-errors when the create_engine creates the dataBase.

```
1  excel_import.py
2
3      import pandas as pd
4      import numpy as np
5      import sqlite3
6      from sqlalchemy import create_engine
7
8
9      def data_import(dataBase:pd.DataFrame, dataTable:str, excelFile:str):
10         '''Function for importing Excel data into a database'''
11
12         # Read the excel file and store the data in a DataFrame
13         excelFile = pd.read_excel(excelFile)
14
15         # Check for non-numeric values in non-first columns and replace them with na points
16         excelFile.iloc[:, 1:] = excelFile.iloc[:, 1:].apply(pd.to_numeric, errors='coerce').fillna(np.nan)
17
18         # Create a connection to the SQLite database (or create it if it doesn't exist)
19         conn = sqlite3.connect(dataBase)
20         cursor = conn.cursor()
21
22         # Drop the table if it exists so the database can be recreated from scratch for testing
23         cursor.execute(f'DROP TABLE IF EXISTS {dataTable}')
24
25         # Use SQLAlchemy to import the DataFrame into the SQL database
26         engine = create_engine(f'sqlite:///{dataBase}')
27         excelFile.to_sql(dataTable, engine, if_exists='replace', index=False)
28
29         # Commit the changes and close the connection
30         conn.commit()
31         conn.close()
32         print('Data imported successfully!')
```

All code will be enclosed in the additional files included

1.2 Extreme Outliers

Using the database as a source of data, we can now begin to pre-process the data. Using matplotlib, we can plot the data to see if there are any trends or patterns in the data, along with any outliers that may need to be removed.

In the given dataset, there were a number of values that were not possible, such as negative flow rates and rainfall values in the thousands. These values need to be removed from the dataset to ensure the model is accurate. To remove these and other 'extreme' outliers, I used the inter-quartile range (IQR) method. When I implemented this, I used a variable to multiply by the IQR to get the upper and lower bounds, allowing me to refine the data to a point I thought was acceptable.

A typical weight for extreme values is 3, this is because the IQR is the range between the 25th and 75th percentile, so multiplying by 3 gives a range of 99.7% of the data (when its normally distributed). However, as the data is skewed heavily towards 0, the 75th percentile still encapsulates a large amount of possible data within a wide range. After experimenting with different values, the results were as follows:

Weight	Amount Removed
1.5	2031
3.0	782
6.0	176
10.0	50

Table 1.1: Results of different weights for IQR

From testing, I decided that 10 was the best weight to use, as it removed the most extreme values without removing too much data. The remaining data looks real and is within a reasonable range for realistic rainfall and flow rates.

1.3 Averaging and Deviations

The data has had its extreme outliers removed, but it is still not in a usable state. The data needs to be averaged and standardised to ensure that the data is in a usable state for the neural network.

> *For context: These functions are called within an iterative loop that acts on them for each of the columns in the database. columnData is a pandas data frame of the date with that dates record.*
>
> *The data points are uniformly iterated with columnData.itertuples(), hence the use of the point[n] convention to access the data.*
>
> *When a data point is removed, it is stored in a dictionary object, with the key being the column name and the value being a dictionary of the removed data points. All code will be enclosed in the additional files included.*

Our IQR with extreme weighting method does not account for large amount of rainfall in a dry spell, or low flow rate at a time of flooding. To account for this, we will use methods to determine if the data is within a reasonable range, over a period of time.

Rainfall and especially flow rate data move in trends, so to best capture the data we will use a condensed version of the data, with only its neighbors. We will calculate our outliers over this as rainfall in summer will not be as high as in winter (it's still England though), so calculating the averages of the whole data set will not be accurate.

data_cleaning.py

```
import numpy as np
import pandas as pd

def local_range_df(columnData:pd.DataFrame, dataPoint:tuple, columnLength:int, windowSize:int) -> pd.DataFrame:
    '''Function to create a smaller range of data around a point'''
    windowSplit = round(windowSize/2)

    # Defines the start and end of the range
    if int(dataPoint[0])-windowSplit < 0:
        '''If the point is close to 0'''
        start = 0
        end = windowSize-int(dataPoint[0])
    elif int(dataPoint[0])+windowSplit > columnLength:
        '''If the point is close to the end of the DF'''
        end = columnLength
        start = columnLength - windowSize
    else:
        '''If the point is in the middle of the DF'''
        start = int(dataPoint[0])-windowSplit
        end = int(dataPoint[0])+windowSplit

    # Create a range of the data around the point
    localData = columnData.loc[start:end]

    return localData
```

We will pass this local data to the standard deviation and IQR functions to calculate the upper and lower bounds of the data.

Flow rate of a river rises and falls in patterns, so to best catch outliers we will use a mean and standard deviation over a period of time.

data_cleaning.py

```
    import numpy as np
    import pandas as pd

    def standard_deviation_calculator(columnData:pd.DataFrame, columnName:str, deviationWeight:int) -> tuple[float, float]:
        '''Function to calculate the standard deviation of a range of data'''

        # Calculate the mean and standard deviation of the DataFrame
        deviation = columnData[columnName].std(axis=0, skipna=True, numeric_only=True, ddof=1)
        average = columnData[columnName].mean(axis=0, skipna=True, numeric_only=True)

        # Calculate the upper and lower bounds
        lowerBound = average - (deviationWeight * deviation)
        upperBound = average + (deviationWeight * deviation)

        return lowerBound, upperBound
```

This method determines if the data follows the trend of its neighbors, and if it does not, it is removed from the dataset. One day of high flow rate isn't a realistic measurement as the river constantly flows.

However, rainfall can be more sporadic, so we will once again use a weighted IQR method to remove extreme values.

data_cleaning.py

```
    import numpy as np
    import pandas as pd

    def iqr_calculator(columnData:pd.DataFrame, columnName:str, deviationWeight:int) -> tuple[float, float]:
        '''Function to calculate the IQR of a range of data'''

        # Calculate the IQR of the range
        Q1 = columnData[columnName].quantile(0.25)
        Q3 = columnData[columnName].quantile(0.75)
        IQR = Q3 - Q1

        # Calculate the upper and lower bounds, round to 3 decimal places
        lowerBound = round(Q1 - (deviationWeight * IQR), 3)
        upperBound = round(Q3 + (deviationWeight * IQR), 3)

        return lowerBound, upperBound
```

This will be done over a period of time, as a single day of high rainfall is possible in a storm, before a low rainfall spell occurs again. By splitting the data this way we allow the more accurate flow rate data to be used in the model, while still allowing for the sporadic nature of rainfall.

Once the data points have been removed from the dataset, we can replace them with the average of the surrounding data points. This is to ensure that the data is continuous and that the model can be trained on the data.

1.4 Interpolation

When the algorithm removes the data points, it leaves gaps in the data. I made this decision to separate the extraction and interpolation for two reasons:

1. To allow for efficiency of interpolation
2. For accuracy, as im only using the data points that were recorded, not interpolated

To interpolate the data, I used a simple moving average (SMA) method to fill the gaps. I implemented this with a variable window size for the SMA to allow for flexibility in the data. The numpy array allows for skipping of NaN values, so the data can be interpolated in places where I have removed the data points. However, too small of a window size ends up with a variation of a forward fill at points with data removed. I decided that a window size of 6 was the best for the data, allowing for trends to show without ending up with a forward fill.

data_cleaning.py

```
    import numpy as np
    import pandas as pd

    def simple_moving_average(columnData:pd.DataFrame, columnName:str, windowSize:int, removedDict:dict, updatedDict:dict) -> tuple[dict, dict]:
        '''Function to update empty points in the dataset'''

```

```
        # Find all NaN values in the column
        emptyPoints = columnData[columnData[columnName].isna()]

        # Iterate through the empty points
        for point in emptyPoints.itertuples():
            '''Iterate through the empty points in the DataFrame'''

            # Get the local range of data around the point
            localData = local_range_df(columnData, point, len(columnData), windowSize)

            # Calculate the average of the range
            average = round(localData[columnName].mean(skipna=True),)

            # Update the statistics
            removedDict[columnName][point[1]].append(float(average))

            # For visualisation purposes, store some of the updated data in separate dictionary
            try:
                updatedDict[columnName][point[1]].append(float(average))
            except:
                None

            # Replace the empty point with the average
            columnData.loc[point[0], columnName] = average

        return removedDict, updatedDict
```

The outputs of this function are two dictionary objects. As explained in the context note, the dictionary is used to store the data points that were removed from the dataset. Both of them are the same, except updatedDict is used for visualisation purposes. As the extreme values are present in the removedDict, when plotted they skew the graph so you can't see the trends in the data.

I then go on to use the updatedDict to plot the data, showing the trends in the data, along with the removed data and the ones that replaced it. This is to show the accuracy of the interpolation and to show the trends in the data. The removedDict is used to update the SQL database, so the data can be used in the neural network; we need a record of data that was changed to be able to update it. If we instead saved it to the data frame, we would lose the location of updated data points. This would result in updating the whole database with the whole data frame, wasting a lot of time and resources replacing points with values they already have. The database has 11,688 data points, with 992 (for variables stated in figure 1.5.2) removed and replaced. That's 8.5% of the data and more than 10 times more efficient only updating those.

1.5 Plotting trends

Once the data has been cleaned and interpolated, we can plot the data to see the trends in the data. We can also make judgments on the variables that determine which point are outliers and which are not.

Lets use the data at Crakehill as an example location for visualisation:

Figure 1.5.1 show us the data at Crakehill, with the unrealistic data present. As I stated before, the data scale is affected by the outliers, so the trends in the data are not visible.

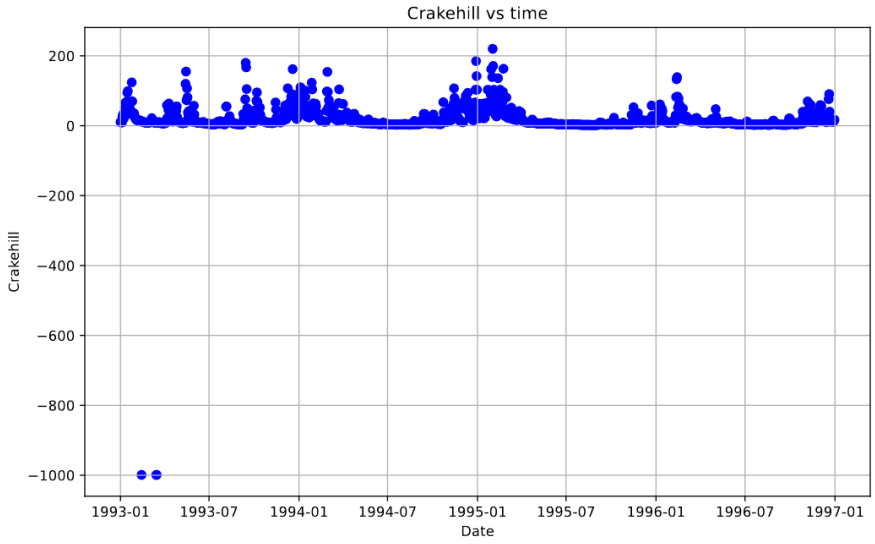


Figure 1.5.1: Crakehill Data with unrealistic data

Figure 1.5.2 shows the data at Crakehill, with the unrealistic data points removed, but still containing outliers

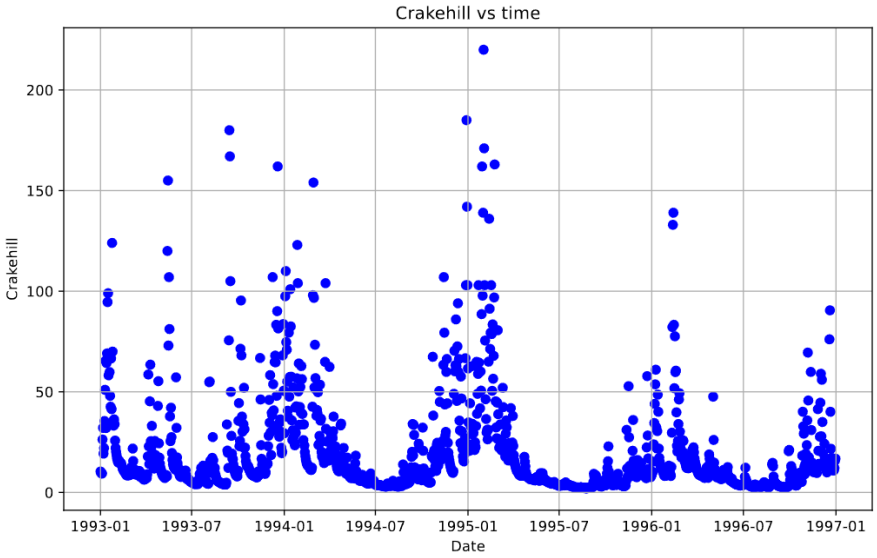


Figure 1.5.2: Crakehill Data without unrealistic data

Finally, figure 1.5.3 shows us the result of our interpolation, with the data points that were removed and replaced. Blue points are the data we are using, red points are the data that was removed and green being what replaced them.

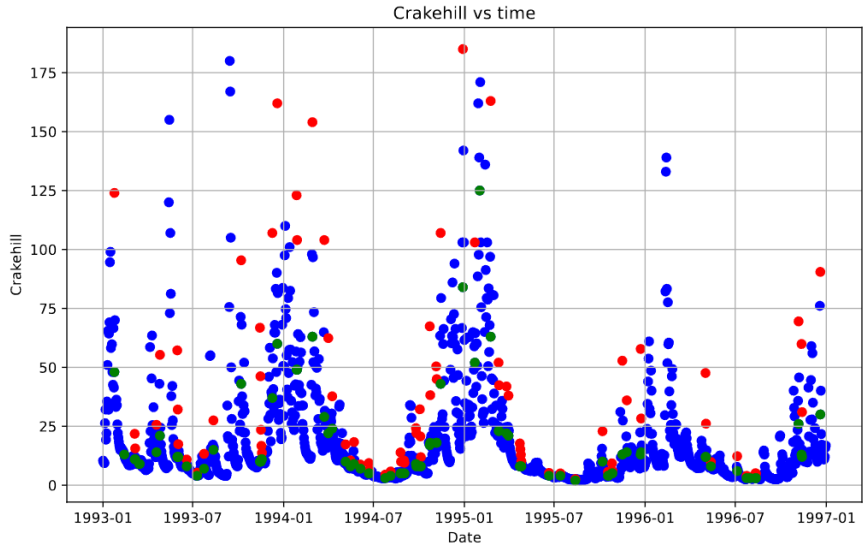


Figure 1.5.3: Crakehill Data interpolated

This specific figure has:

- IQR weight of 1.5
- Standard deviation weight of 2
- Window size of 10 for removal
- Window size of 6 for interpolation
- 91 data points removed and replaced (992 across all locations)

When looking at figure 1.5.2, we can see that in 1993, the flow rate data was much more sporadic than the following years, then in figure 1.5.3, the data is clearly less sporadic and follows the trends more closely. This data should now be considered usable to train our Multi-Layer Perceptron (MLP) model. The database can now be updated with the new data.

1.6 Correlation

Chapter 2

Implementing the Neural Network

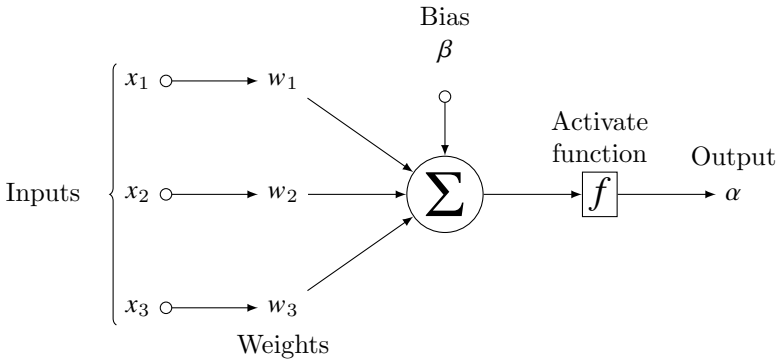
The network will be implemented with a back propagation learning algorithm.

2.1 Data Splitting

50 25 25

2.2 Model Architecture

nueron architecutre and activation functions



2.3 Activation Functions

2.4 Loss Functions

2.5 Network with 1 hidden layer

The first implementation of the neural network will be a simple 1 hidden layer network. This is to refine parameters and theory before applying to multiple nodes and layers.

2.5.1 Forward Pass

For n inputs, m hidden nodes and q output nodes.

The input matrix takes the n data points from the training data in the form:

$$\text{Matrix } \mathbf{I} \rightarrow [1 \times n] : \mathbf{I} = \begin{vmatrix} \mathbf{I}_1, & \mathbf{I}_2, & \cdots, & \mathbf{I}_n \end{vmatrix} \tag{2.5.1}$$

The weight matrix ω is a $n \times m$ matrix, with each row representing the weights of a input nodes to each connected hidden node.

$$\text{Matrix } \omega \rightarrow [n \times m] : \omega = \begin{vmatrix} \omega_{11}, & \omega_{12}, & \cdots, & \omega_{1m} \\ \omega_{21}, & \ddots & & \omega_{2m} \\ \vdots & & \ddots & \vdots \\ \omega_{n1}, & \omega_{n2}, & \cdots, & \omega_{nm} \end{vmatrix} \tag{2.5.2}$$

The matrix multiplication of the input matrix and the weight matrix gives us a matrix of dimensions $1 \times m$. This correlates

to the amount of hidden nodes in the network, a sum of weights multiplied by their corresponding input data for each hidden node.

$$\text{Matrix } \mathbf{I} \cdot \omega \rightarrow [1 \times n] \cdot [n \times m] \rightarrow [1 \times m] : \mathbf{I} = \cdot\omega \left| \mathbf{I}_1 \cdot \omega_{11} + \cdots + \mathbf{I}_n \cdot \omega_{n1}, \quad \cdots, \mathbf{I}_m \cdot \omega_{1m} + \cdots + \mathbf{I}_n \cdot \omega_{nm} \right| \tag{2.5.3}$$

Each node in the hidden layer will have a bias term β. Formatting this as a matrix, we get a $1 \times m$ matrix, which aligns with the dimensions matrix $\mathbf{I} \cdot \omega$ (2.5.3).

$$\text{Matrix } \beta \rightarrow [1 \times m] : \beta = \left| \beta_1, \quad \beta_2, \quad \cdots, \quad \beta_m \right| s \tag{2.5.4}$$

We apply an element-wise summation of the matrix $\mathbf{I} \cdot \omega$ and the bias matrix β, to find the matrix $\mathcal{S}^1 \rightarrow [1 \times m]$: the summation of each node in layer 1. Applying the activation function $\mathcal{F}_1$ to the summation matrix $\mathcal{S}$ gives us the output α of the hidden layer.

Note that the activation function is labeled $\mathcal{F}_1$ as it is the first activation function in the network, and can differ from other activation functions in the network.

$$\text{Matrix } \alpha \rightarrow [1 \times m] : \alpha = \mathcal{F}_1(\mathcal{S}) = \mathcal{F}_1(\mathbf{I} \cdot \omega + \beta) \tag{2.5.5}$$

This activation function has applied to every element in the matrix. This matrix α is then used as the input for the output layer of the network.
The second weight matrix μ is a $m \times q$ matrix, with each row representing the weights of the hidden nodes to the output nodes.

$$\text{Matrix } \mu \rightarrow [m \times q] : \mu = \begin{vmatrix} \mu_{11}, & \mu_{22}, & \cdots, & \mu_{1q} \\ \mu_{21}, & \ddots & & \mu_{2q} \\ \vdots & & \ddots & \vdots \\ \mu_{m1}, & \mu_{m2}, & \cdots, & \mu_{mq} \end{vmatrix} \tag{2.5.6}$$

Just like our inputs I and weights ω, the matrix multiplication of the hidden layer output α and the weight matrix μ gives us a matrix of dimensions $1 \times q$. This correlates to the amount of nodes in the output layer, where each object in the matrix is the sum of the weight that connect the output node to its hidden nodes, multiplied by their hidden node output.

$$\text{Matrix } \alpha \cdot \mu \rightarrow [1 \times m] \cdot [m \times q] \rightarrow [1 \times q] : \alpha \cdot \mu = \left| \alpha_1 \cdot \mu_{11} + \cdots + \alpha_m \cdot \mu_{m1}, \quad \cdots, \alpha_1 \cdot \mu_{1q} + \cdots + \alpha_m \cdot \mu_{mq} \right| \tag{2.5.7}$$

Matrix ρ is the bias matrix for the output layer, with dimensions $1 \times q$.

$$\text{Matrix } \rho \rightarrow [1 \times q] : \rho = \left| \rho_1, \quad \rho_2, \quad \cdots \quad \rho_q \right| \tag{2.5.8}$$

We apply an element-wise summation of the matrix $\alpha \cdot \mu$ and the bias matrix ρ, to find the matrix $\mathcal{S}^2 \rightarrow [1 \times q]$ summation of the nodes in layer 2. Applying the activation function $\mathcal{F}_2$ to the summation matrix $\mathcal{S}^2$ gives us the output Output of the network.

Note that $\mathcal{F}_2$ the second activation function in the network, and can differ from $\mathcal{F}_1$.

$$\text{Output } \rightarrow [1 \times q] : O_q = \mathcal{F}_2(\mathcal{S}^2) = \mathcal{F}_2(\alpha \cdot \mu + \rho) \tag{2.5.9}$$

This gives us our prediction from the forward pass through the network.

2.5.2 Backward Pass

The backward pass is used to update the weights and biases of the network, to reduce the error of the network and improve the accuracy of the model. Back propagation is about finding how much of the error is due to each weight in the network, and then updating the weights to reduce the error.

Using partial derivatives....

the partial derivative of the output nodes, with m layers, is given by:

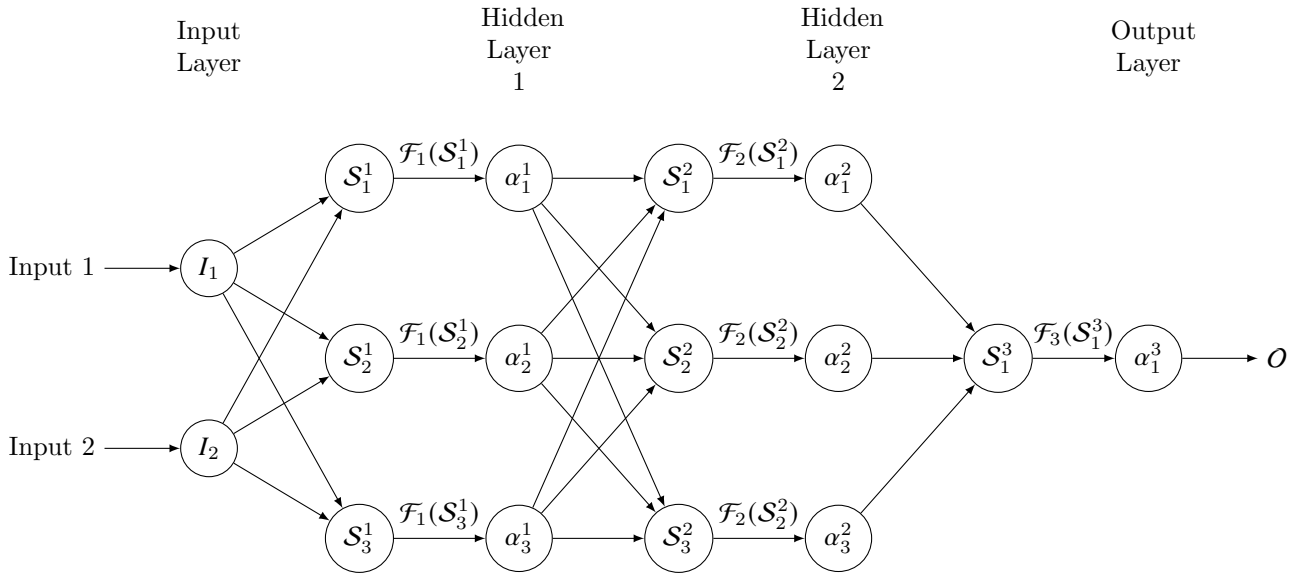
$$\delta_n^m == \mathcal{L}(C_n, O_n) \cdot \mathcal{F}'_m(\mathcal{S}_n^m) \tag{2.5.10}$$

Where $\mathcal{L}(C_n, O_n)$ is the loss function, C_n is the correct output and O_n is the predicted output of the node.

For the hidden nodes in layers $1 \leq k < m$:

$$\delta_n^k == \sum_{q=1}^{p} \left(\omega_{nq} \cdot \delta_q^{k+1} \cdot \mathcal{F}'_k(\mathcal{S}_q^{k+1}) \right) \tag{2.5.11}$$

With n nodes in layer k, p nodes in layer $k+1$ and ω_{nq} being the weight between node n in layer k and node q in layer $k+1$.



$$\omega'^k_{nq} = \omega'^k_{nq} + \eta \cdot \delta_q^{k+1} \cdot \alpha_n \tag{2.5.12}$$

$$\beta'^k_{nq} = \beta'^k_{nq} + \eta \cdot \delta_q^{k+1} \tag{2.5.13}$$